

Document de validation

Projet GL51

Équipe :

AZMAOUI Ikhlas,
BENKHADRA Widad,
BENNASER Ahmed,
LRHOFI Mohammed Akram,
MARCHAND Thibault.

Institution : Ensimag (Grenoble INP)

Date : Janvier 2026

Encadré par :

GABRIELA NICOLE GONZALEZ SAEZ

Table des matières

1	Description et organisation des tests	2
1.1	Types de tests pour chaque test/ passe	2
1.1.1	Tests syntaxiques et lexicaux	2
1.1.2	Tests contextuels	2
1.1.3	Tests de génération de code	3
1.2	Organisation de la banque de tests	3
1.3	Objectifs et atteinte des objectifs	3
2	Scripts de tests et automatisation	3
3	Gestion des risques et des rendus	4
3.1	Gestion des risques	4
3.1.1	Gestion des risques humains	4
3.1.2	Gestion des risques liés aux outils	4
3.1.3	Gestion des risques organisationnels	5
3.1.4	Gestion des risques techniques	5
4	Résultats Jacoco et couverture	6
5	Méthodes de validation utilisées autres que le test	9

1 Description et organisation des tests

La validation du compilateur Deca repose sur une banque de tests construite de manière progressive tout au long du projet, en parallèle de l'implémentation des différentes passes du compilateur. Cette stratégie incrémentale permet de valider chaque fonctionnalité dès son développement et de garantir la non-régression lors des évolutions ultérieures.

Les tests sont organisés à la fois par **type de validation** (syntaxe, contexte, génération de code) et par **nature** (tests valides et tests invalides), conformément aux consignes du polycopié.

1.1 Types de tests pour chaque test/ passe

1.1.1 Tests syntaxiques et lexicaux

Ces tests valident la **phase A** du compilateur, correspondant à l'analyse lexicale (reconnaissance des tokens) et à l'analyse syntaxique (respect de la grammaire Deca). Ils permettent de vérifier :

- l'acceptation des programmes syntaxiquement corrects ;
- la détection précise des erreurs lexicales et syntaxiques ;
- l'arrêt de la compilation dès la première erreur bloquante.

Tests valides D'après ces tests, nous avons essayé de couvrir les programmes minimaux (programme vide), les littéraux (entiers, flottants, chaînes, caractères), les identificateurs valides, les instructions simples (affectation, `print`, `println`), les opérateurs arithmétiques, logiques et de comparaison, les structures de base (blocs, déclarations avant instructions) et les imports. Tout ceci dans le but de vérifier que tout programme conforme à la grammaire Deca est accepté sans erreur.

Tests invalides Ces tests couvrent plutôt les erreurs syntaxiques comme parenthèses manquantes, point-virgule absent ou mal placé, accolades manquantes, mauvaise structure des blocs, opérateurs mal utilisés, etc... et lexicales comme identificateurs mal formés, constantes numériques invalides, caractères interdits, chaînes non terminées, mauvais échappements... Tout ceci pour s'assurer que le compilateur détecte correctement les erreurs et produit des messages cohérents, sans poursuivre l'analyse.

1.1.2 Tests contextuels

Les tests contextuels valident la passe de vérification des règles contextuelles, avec et sans objets.

Tests valides Afin de garantir que tout programme respectant les règles contextuelles est accepté, ces tests vérifient la déclaration préalable des identificateurs, la cohérence des types (affectations, expressions, conversions implicites), l'utilisation correcte des booléens dans les conditions, la validité des expressions arithmétiques et logiques, les appels de méthodes et la gestion des paramètres, l'héritage, le cast valide, l'utilisation de `this` et `instanceof` et les règles de visibilité (`protected`, accès dans les sous-classes).

Tests invalides Pour vérifier la détection exhaustive des violations des règles contextuelles, avec un arrêt propre de la compilation, nous avons créé les tests principaux suivant : variables non déclarées ou doublement déclarées, types inconnus ou incompatibles, affectations illégales (lvalue invalide, type void, cast impossible), erreurs sur les conditions (if, while non booléens), erreurs liées aux méthodes (signature, retour, paramètres), erreurs liées aux objets (héritage invalide, accès interdit, this dans le main).

1.1.3 Tests de génération de code

Dans le but de s'assurer que le code généré est correct, efficace et conforme au comportement attendu à l'exécution. Ces tests couvrent :

- les expressions arithmétiques et logiques complexes ;
- les structures de contrôle (if, while, blocs imbriqués) ;
- la gestion des registres, de la pile et des offsets globaux ;
- les entrées/sorties (readInt, print, println) ;
- les erreurs à l'exécution (division par zéro, débordement, tas plein) ;
- les mécanismes objets : allocation, champs, méthodes, héritage, polymorphisme, cast et instanceof.

1.2 Organisation de la banque de tests

Les tests sont organisés dans l'arborescence standard du projet comme mentionné dans le polycopié. Chaque test correspond à un cas précis, clairement identifié par son nom et sa description, facilitant : la compréhension du but du test, la reproduction des erreurs et l'ajout de nouveaux tests ciblés.

1.3 Objectifs et atteinte des objectifs

Nombreux sont les objectifs de notre stratégie de tests : on retrouve principalement la couverture de l'ensemble des fonctionnalités du langage Deca, la détection des erreurs le plus tôt possible dans la chaîne de compilation, et l'affichage de messages d'erreurs corrects à l'utilisateur.

Ces objectifs ont été atteints grâce à :

- une construction progressive de la banque de tests,
- une séparation claire entre tests valides et invalides,
- une couverture systématique des règles du langage énoncées dans polycopié,
- l'ajout de tests spécifiques pour les cas limites et les parties critiques (pressions registres, récursion, erreurs d'exécution).

2 Scripts de tests et automatisation

La validation du compilateur repose sur un ensemble de scripts automatisés permettant d'exécuter l'intégralité de la banque de tests de manière reproductible. Ces scripts couvrent les différentes passes du compilateur (syntaxe, contexte, génération de code) et assurent

une exécution homogène des tests valides et invalides grâce à la commande **mvn test** ou **mvn verify**.

Les tests sont exécutés via des scripts shell (**basic_synt.sh**, **basic_lex.sh**, et **basic_gencode.sh**) fournis par les profs et complétés par nous mêmes dans le dossier **src/test/script** pour qu'ils puissent parcourir automatiquement toute l'arborescence des tests .deca et lancent le compilateur Deca avec les options adaptées à chaque type de test.

Pour les tests :

- valides : la compilation doit se terminer sans erreur,
- invalides : la compilation doit échouer et produire une erreur conforme aux règles contextuelles ou syntaxiques attendues.

Ajout de nouveaux tests

L'ajout d'un nouveau test ne nécessite aucune modification des scripts, il suffit d'ajouter le fichier .deca dans le dossier approprié (valid ou invalid), le test sera automatiquement pris en compte lors de la prochaine exécution.

3 Gestion des risques et des rendus

3.1 Gestion des risques

Lors du déroulement du projet, plusieurs risques ont été identifiés. On peut les diviser en plusieurs catégories distinctes, avec pour chacune des mesures préventives et correctives propres.

3.1.1 Gestion des risques humains

S'agissant des risques humains, nous avons identifié les risques suivants :

- **Absences, indisponibilités, éloignement géographique** : Une partie du projet s'est déroulée durant, et immédiatement suivant les vacances scolaires. De plus, des empêchements à certaines séances de travail en groupe étaient inévitables. Pour pallier l'éventuel impact sur le projet, nous avons mis en place de la documentation partagée et régulièrement mise à jour, nous avons communiqué régulièrement (principalement par le biais de Whatsapp et Google Meet), et nous avons travaillé à plusieurs sur les tâches critiques du projet.
- **Conflits internes** : Comme pour tout projet de taille, des conflits entre les membres du groupe risquent d'apparaître à cause du stress, des attentes différentes et du ressenti de chacun quant à la charge de travail individuelle et de la qualité du projet. Pour anticiper et pallier ces tensions internes, nous avons communiqué de manière bienveillante, respectant notre charte d'équipe et partageant des retours constructifs à chaque fois que cela était possible.

3.1.2 Gestion des risques liés aux outils

S'agissant des risques liés aux outils, nous avons identifié les risques suivants :

- **Perte de code, écrasement des modifications, dégradation de fonctionnalités existantes** : lors des premières phases du projet, nous avons travaillé au

maximum sur des branches protégées afin d'éviter les conflits. Dès que les fonctionnalités ajoutées semblaient fonctionnelles et ne cassaient rien qui existait déjà, nous faisons des merges vers la branche principale, de manière régulière. Durant la dernière phase du projet, une partie du groupe travaillait sur une branche dénuée de problèmes, afin d'ajouter quelques classes manquantes et d'observer régulièrement la progression des pourcentages de couverture des tests, tandis que l'autre partie s'est concentrée sur la résolution de bugs sur la branche principale.

- **Problèmes de configuration IDE** : afin de pallier les éventuels problèmes liés aux configurations versionnées (.vscode/, .idea/) nous avons utilisé maven en ligne de commande.

3.1.3 Gestion des risques organisationnels

S'agissant des risques organisationnels, nous avons identifié les risques suivants :

- **Retards par rapport aux rendus** : Notre progression dans le projet était largement guidée et articulée autour des attendus des différents rendus. Pour le rendu intermédiaire, lors duquel on devait présenter un compilateur fonctionnel sur le sans-objet ainsi qu'une banque de tests aussi complète que possible sur cette partie, nous avons constaté un certain retard dans notre progression. Nous avons donc mis en place des points d'avancement quotidiens ainsi que des séances de travail quasi-quotidiennes à l'Ensimag afin d'augmenter en efficacité de travail et en rapidité de communication.

3.1.4 Gestion des risques techniques

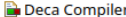
S'agissant des risques techniques, nous avons identifié les risques suivants :

- **Bugs en génération de code** : Ce risque était le plus impactant pour le projet, car les erreurs de génération de code peuvent produire des programmes incorrects tout en restant syntaxiquement et contextuellement valides. Elles sont souvent difficiles à détecter, car elles n'apparaissent qu'à l'exécution. Pour pallier ce risque, nous avons mis en place :
 - des tests unitaires systématiques ciblant la génération de code, couvrant progressivement les expressions, les structures de contrôle et les cas limites ;
 - une validation incrémentale, consistant à tester chaque nouvelle fonctionnalité dès son implémentation afin d'éviter l'accumulation d'erreurs ;
 - l'utilisation de l'IMA pour analyser le code assembleur généré et localiser précisément les anomalies ;
 - une relecture manuelle du code généré pour les programmes complexes ou critiques.
- **Tests insuffisants ou incorrectement implémentés** : Afin d'utiliser efficacement les tests pour tester le bon fonctionnement et aussi révéler des lacunes dans le code, nous nous sommes beaucoup basés sur l'outil Jacoco en augmentant graduellement la couverture globale du code. Nous avons également essayé de garder un équilibre entre les tests valides, qui vérifiaient que les fonctionnalités basiques étaient implémentées, et les tests invalides, qui vont jusque dans les éléments les plus subtils du langage et se veulent aussi exhaustifs que possible.

4 Résultats Jacoco et couverture

La couverture de code du compilateur a été évaluée à l'aide de l'outil JaCoCo, intégré au processus de build Maven. Cette analyse permet de mesurer la part du code réellement exécutée lors du passage de la base de tests, et d'identifier les zones insuffisamment couvertes.

Ci-dessous, les résultats de l'outil Jacoco :


Decca Compiler


[Sessions](#)

Decca Compiler

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
fr.ensimag.decca.syntax		76%		58%	460	674	470	2,029	248	375	3	51
fr.ensimag.decca.tree		83%		73%	181	651	339	1,776	99	453	1	86
fr.ensimag.decca		54%		38%	48	83	113	235	8	38	2	5
fr.ensimag.ima.pseudocode		79%		75%	24	84	37	180	19	74	1	26
fr.ensimag.decca.context		89%		87%	26	142	29	239	19	111	0	21
fr.ensimag.decca.tools		35%		33%	10	16	27	39	8	13	1	3
fr.ensimag.ima.pseudocode.instructions		75%		n/a	17	62	29	111	17	62	12	54
fr.ensimag.decca.codegen		96%		72%	5	38	3	90	0	29	0	6
Total	4,319 of 20,018	78%	398 of 1,120	64%	771	1,750	1,047	4,699	418	1,155	20	252

FIGURE 1 – Pourcentages de couverture pour le compilateur

Les résultats JaCoCo indiquent une bonne couverture globale du compilateur (78 % des instructions et 64 % des branches). Les packages clés syntax (76 %), context (89 %), codegen (96 %), et tree (83 %) sont bien couverts, ce qui montre que les parties centrales du compilateur sont largement exercées par les tests. Les couvertures plus faibles observées dans syntax et tools s'expliquent principalement par des branches d'erreur ou des cas exceptionnels, plus difficiles à déclencher, mais n'impactent pas le fonctionnement principal du compilateur.

Deca Compiler > fr.ensimag.deca.tree

[Source Files](#) [Sessions](#)

fr.ensimag.deca.tree

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
DeclMethod		76%		61%	8	18	20	90	2	9	0	1
Tree		67%		81%	8	30	19	68	6	22	0	1
Identifier		78%		87%	4	28	19	84	2	20	0	1
Cast		80%		64%	5	13	12	53	1	6	0	1
DeclField		76%		60%	8	16	15	54	5	11	0	1
MethodCall		87%		66%	6	18	11	79	2	9	0	1
ReadFloat		0%		0%	7	7	12	12	6	6	1	1
AbstractBinaryExpr		80%		50%	8	19	7	47	1	9	0	1
AbstractExpr		87%		82%	8	29	10	72	3	15	0	1
InstanceOf		85%		50%	3	8	10	50	1	6	0	1
ListDeclMethod		54%		50%	4	9	9	18	2	5	0	1
DeclClass		93%		86%	4	21	11	121	1	10	0	1
ListExpr		64%		58%	4	9	9	22	1	3	0	1
Selection		87%		66%	7	16	6	48	1	7	0	1
While		79%		100%	3	9	10	39	3	8	0	1
IfThenElse		81%	n/a	n/a	1	6	8	37	1	6	0	1
ListDeclParam		71%		60%	3	10	8	24	1	5	0	1
DeclVar		84%		100%	1	8	7	40	1	6	0	1
DeclParam		83%		75%	3	10	7	40	2	8	0	1
AbstractPrint		80%		87%	2	16	8	33	1	8	0	1
New		87%		66%	3	9	6	41	1	6	0	1
Main		86%	n/a	n/a	1	7	7	39	1	7	0	1
MethodBody		80%		100%	1	8	7	28	1	7	0	1
StringLiteral		72%		50%	4	11	6	19	2	9	0	1
ListDeclVar		66%		66%	2	7	5	14	1	4	0	1
ListDeclClass		88%		77%	4	18	5	42	1	9	0	1
ListDeclField		75%		75%	2	9	5	18	1	5	0	1
ListInst		67%		66%	2	7	5	14	1	4	0	1
MethodAsmBody		72%		50%	2	7	4	17	1	6	0	1
Program		88%	n/a	n/a	3	9	5	38	3	9	0	1
Return		78%		100%	2	8	5	21	2	7	0	1
Not		78%		75%	2	7	2	13	1	5	0	1
UnaryMinus		71%		75%	2	6	2	10	1	4	0	1
Assign		88%		75%	3	10	3	25	2	8	0	1
This		81%		50%	3	10	4	18	2	8	0	1
BooleanLiteral		83%		100%	2	11	3	16	2	9	0	1
LocationException		82%		50%	3	7	3	15	0	4	0	1
AbstractUnaryExpr		78%	n/a	n/a	1	6	3	15	1	6	0	1
TreeList		91%		100%	2	14	2	23	2	11	0	1
Initialization		88%	n/a	n/a	1	8	3	23	1	8	0	1
FloatLiteral		85%		50%	3	10	2	14	1	8	0	1
Or		85%		100%	2	5	2	12	2	4	0	1
IntLiteral		88%	n/a	n/a	1	9	2	16	1	9	0	1
NoOperation		45%	n/a	n/a	3	6	4	7	3	6	0	1
ReadInt		87%		50%	3	7	3	12	2	6	0	1
Location		91%		50%	2	8	2	14	0	6	0	1
Null		85%	n/a	n/a	1	6	2	9	1	6	0	1
AbstractInst		42%	n/a	n/a	1	2	2	3	1	2	0	1
And		93%		100%	2	5	2	13	2	4	0	1
Multiply		97%		100%	1	8	1	16	1	3	0	1
Modulo		96%		66%	3	7	1	14	1	4	0	1
Divide		96%		87%	2	7	1	14	1	3	0	1
ConvFloat		90%	n/a	n/a	1	4	1	7	1	4	0	1
PrintIn		87%	n/a	n/a	1	3	1	6	1	3	0	1
Plus		86%	n/a	n/a	1	3	1	5	1	3	0	1
Total	1,145 of 6,850	83%	104 of 396	73%	181	651	340	1,781	99	453	1	86

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Minus		86%		n/a	1	3	1	5	1	3	0	1
Lower		83%		n/a	1	3	1	4	1	3	0	1
Equals		83%		n/a	1	3	1	4	1	3	0	1
GreaterOrEqual		83%		n/a	1	3	1	4	1	3	0	1
Greater		83%		n/a	1	3	1	4	1	3	0	1
NotEquals		83%		n/a	1	3	1	4	1	3	0	1
LowerOrEqual		83%		n/a	1	3	1	4	1	3	0	1
Print		71%		n/a	1	2	1	3	1	2	0	1
Empty/Main		88%		n/a	1	7	1	7	1	7	0	1
NoInitialization		88%		n/a	1	7	1	7	1	7	0	1
AbstractOpCmp		100%		93%	1	11	0	28	0	3	0	1
AbstractOpArith		100%		92%	1	9	0	21	0	2	0	1
AbstractOpBool		100%		n/a	0	3	0	16	0	3	0	1
Visibility		100%		n/a	0	1	0	3	0	1	0	1
AbstractMethodBody		100%		n/a	0	3	0	4	0	3	0	1
Tree.new TreeFunction() {...}		100%		n/a	0	2	0	3	0	2	0	1
Tree.new TreeFunction() {...}		100%		n/a	0	2	0	3	0	2	0	1
AbstractOpExactCmp		100%		n/a	0	1	0	2	0	1	0	1
AbstractOpIneq		100%		n/a	0	1	0	2	0	1	0	1
AbstractDeclMethod		100%		n/a	0	1	0	1	0	1	0	1
AbstractProgram		100%		n/a	0	1	0	1	0	1	0	1
AbstractDeclField		100%		n/a	0	1	0	1	0	1	0	1
AbstractReadExpr		100%		n/a	0	1	0	2	0	1	0	1
AbstractLValue		100%		n/a	0	1	0	1	0	1	0	1
AbstractDeclClass		100%		n/a	0	1	0	1	0	1	0	1
AbstractDeclParam		100%		n/a	0	1	0	1	0	1	0	1
AbstractStringLiteral		100%		n/a	0	1	0	1	0	1	0	1
AbstractDeclVar		100%		n/a	0	1	0	1	0	1	0	1
AbstractIdentifier		100%		n/a	0	1	0	1	0	1	0	1
AbstractInitialization		100%		n/a	0	1	0	1	0	1	0	1
AbstractMain		100%		n/a	0	1	0	1	0	1	0	1
Total	1,145 of 6,850	83%	104 of 396	73%	181	651	340	1,781	99	453	1	86

Created with JaCoCo 0.8.13.202504020838

5 Méthodes de validation utilisées autres que le test

Afin d'augmenter la fiabilité du compilateur et de détecter des erreurs difficiles à couvrir uniquement par des cas de test, nous avons veillé à faire les trucs suivants :

- **Relecture et inspection du code :**

Une relecture systématique du code a été effectuée après l'implémentation de chaque passe du compilateur pour vérifier la conformité de l'implémentation avec les règles du polycopié, en plus du contrôle de la cohérence des structures de données (tables des symboles, environnements, types), et la détection des erreurs de logique non immédiatement révélées par les tests.

- **Comparaison avec les spécifications du polycopié :**

Chaque fonctionnalité implémentée a été systématiquement confrontée aux règles formelles définies dans le polycopié : numérotation des règles, contraintes sur les types, comportements attendus en cas d'erreur.

- **Validation par traçage et messages de débogage :**

L'utilisation de l'outil IMA a été particulièrement utile lors des phases de débogage et de détection d'erreurs. L'analyse du code assembleur généré et des messages produits par IMA a permis de localiser précisément l'origine des erreurs, notamment au niveau de la génération de code, de la gestion des registres et de la pile.

- **Validation par décompilation :**

Afin de vérifier la complétude et la fidélité de la traduction du programme source. Le principe consiste à compiler un programme Deca valide, puis appliquer la décompilation sur le code intermédiaire ou généré, et finalement comparer le programme obtenu avec le programme source initial.